

Ledger: A Version Control System for RL Environments

Author: Upendra Kumar

Reference docs:

- [Harbor](#) — what we package environments with today
- [jj backend storage](#)

1. Problem and context

At Proximal, we track changes to environments using git, with repositories hosted on GitHub. We use Github API a lot because we have many automations running in the background that are making changes to our environments (repos).

We run rollouts on our environments (currently packaged with Harbor), QA those rollouts, make changes to the environment and then repeat the same cycle 5-50 times before an environment is actually ready. We also often "fork" an environment into other environments that have small differences with it. And for each environment, we set up a Github action that triggers building the environment and syncing data into our platform.

We've been rate-limited by Github in the past, and we almost certainly will face such issues again, especially as we scale the volume of environments that we produce.

The requirement is therefore to "build a version control system outside of GitHub that gives us the capabilities we need".

2. Requirements

2.1 Functional - what it does

Version Environments: Commit a change to an environment, list its history, diff two versions, restore an earlier one.

Branching / Forking: Branch for parallel experiments; fork an environment into a variant that differs slightly.

Large Artifact Handling: Environments contain datasets, ML models and container images. Editing part of a large file must not re-store the whole file.

Multi-Container Support: An environment may hold many containers. An old version must bring back exactly the containers it was committed with.

Read at scale: Read one file without cloning anything. No file-size limit, no cap on directory width.

Scoped authorization: Grant subsets of operations over subsets of environments — read but not write, fork but not build, these environments and not those.

Build & Sync: Trigger a build and sync into the platform, replacing the per-environment GitHub Action.

Format Agnostic: Store whatever an environment is made of. Harbor is today's packaging; the storage layer must not know that.

2.2 Non-Functional - how well

Concurrent Automations: Many automations run in the background making changes to environments. Work on one environment should never wait on work in another, and two automations colliding should get a clear error rather than a broken result.

Deduplication: Deduplicate data as much as possible. Identical content should be stored once, whether or not the environments are related.

Stability: A retried request must not duplicate. Damaged data must be detected, not served. Losing one component must not take the system down.

Native for Agents: Generation agents already know `git` and `jj` - adopt `jj`'s model rather than inventing vocabulary.

3. Capacity planning

Assumptions

Input	Value
Environments stored	10M, reached over 10 years (~1M/yr)
Actively iterated at any time	0.1M (1%)
Versions per environment	50
Ephemeral branches per active environment	10
Concurrent rollouts	1M
Rollout duration	~30 min — materialize, run the agent, run the verifier, record the result
Unique bytes retained per environment	~2 GiB across its 50-version life

Derived

Quantity	Value	From
Read-only, finished environments	9.9M (99%)	10M – 0.1M
Live refs	~11M	10M defaults + 0.1M × 10 experiments
Commits, lifetime	500M	10M × 50
Corpus	~20 PB, ~21B objects	10M × 2 GiB, ~1 MiB chunks
Commit rate	1.6/s → ~2/s (16/s at 10× peak)	500M ÷ 10 yr
Materialization rate	556/s	1M concurrent ÷ 30 min
Object read rate	~14,000/s , ~8 GiB/s	556/s × ~25 objects each

14,000 reads against 2 writes is roughly **7,000 : 1**. Two conclusions follow:

- **On the read side, throughput is everything.** means the file / objects that never changes should be immutable and content-addressed, that can be cached at any tier, in any region, forever —

with no invalidation and no way to serve a wrong answer.

- **On the write side, throughput is irrelevant and correctness is everything.** At 2 commits per second, throughput is not the constraint; correctness under concurrent automated writers is. Clean conflicts, safe retries, and no lost updates are what matter.

The two sides want opposite things; a design that serves them the same way gets both wrong.

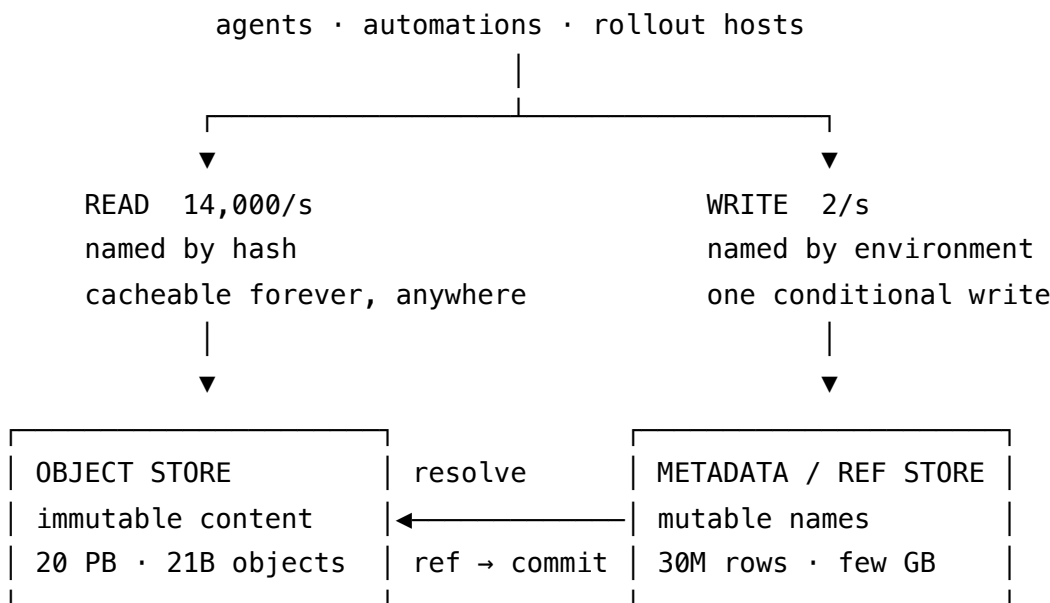
4. Solution overview

4.1 Introducing Ledger

Ledger is a version control system for RL environments, built as a service rather than a tool. An **environment** (a Github repo today) — is a directory tree of task code, datasets, ML models and container images and Ledger keeps every version of it.

Ledger splits storage by data mutability into two layers:

- **Immutable content:** commits, trees, blocks and chunks — is stored in the **object store**, which provides read scalability and high availability.
- **Mutable names:** environments and refs — are stored in the **metadata / ref store** (a transactional store), which guarantees correctness.



This split lays the foundation for meeting the system demands laid out in Section 3.

Most compromises in git hosting come from serving both from one store.

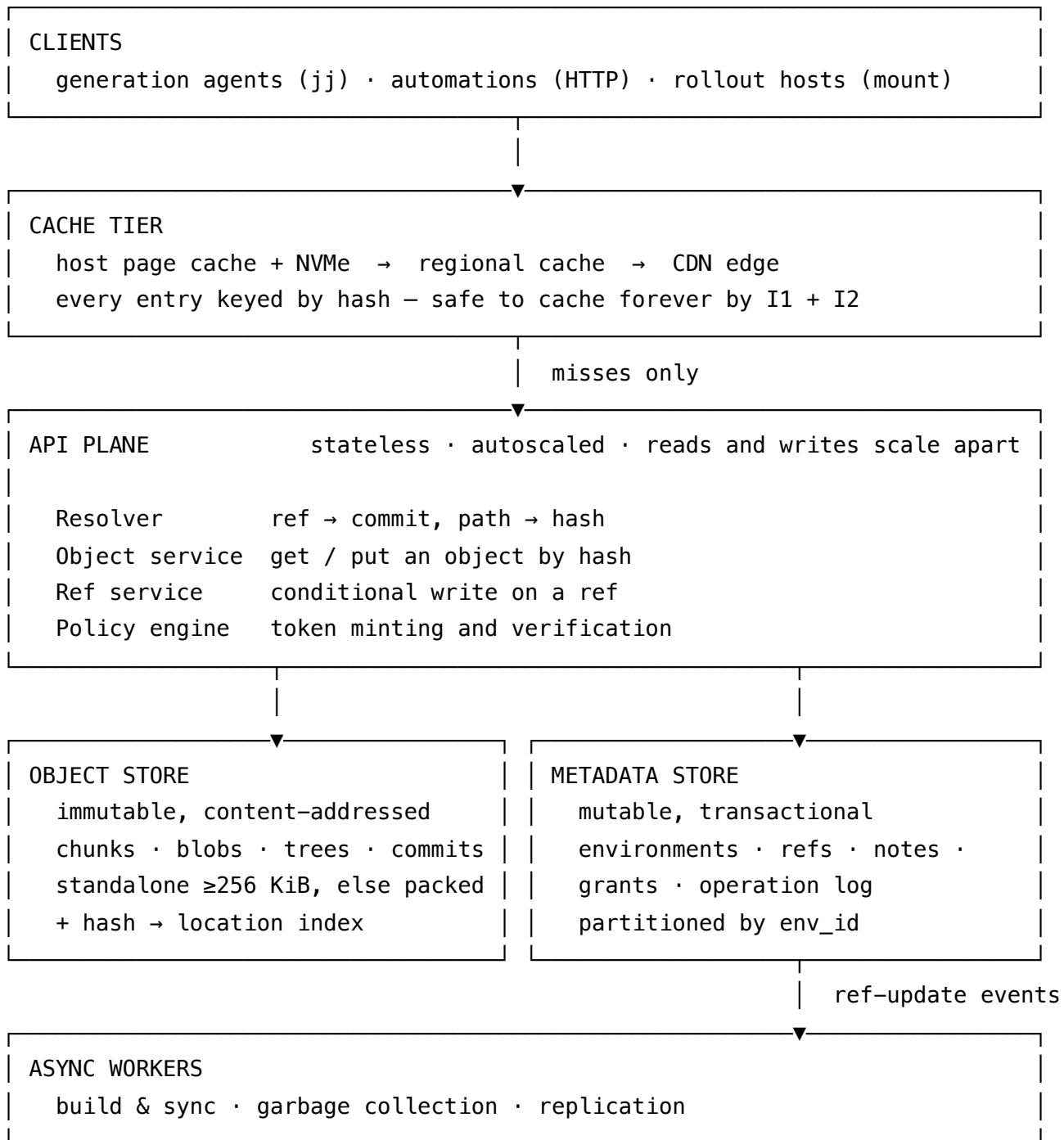
4.2 System invariants

The following five statements hold everywhere in Ledger.

	Invariant	How it helps
I1	An object's name <i>is</i> the hash of its bytes. Same bytes → same name, everywhere, forever	Global dedup and cacheability
I2	A stored object is never modified. A change produces a new object	Caches never need invalidation
I3	A ref only moves to a commit whose entire reachable object set is already durable	A version is never half-written, so a reader never sees a broken environment
I4	Every mutation is a compare-and-swap within one environment's partition	No lost updates, no partially applied writes
I5	An operation replayed with the same idempotency key yields the same result and no further effect	A retried request cannot create a duplicate

4.3 Components

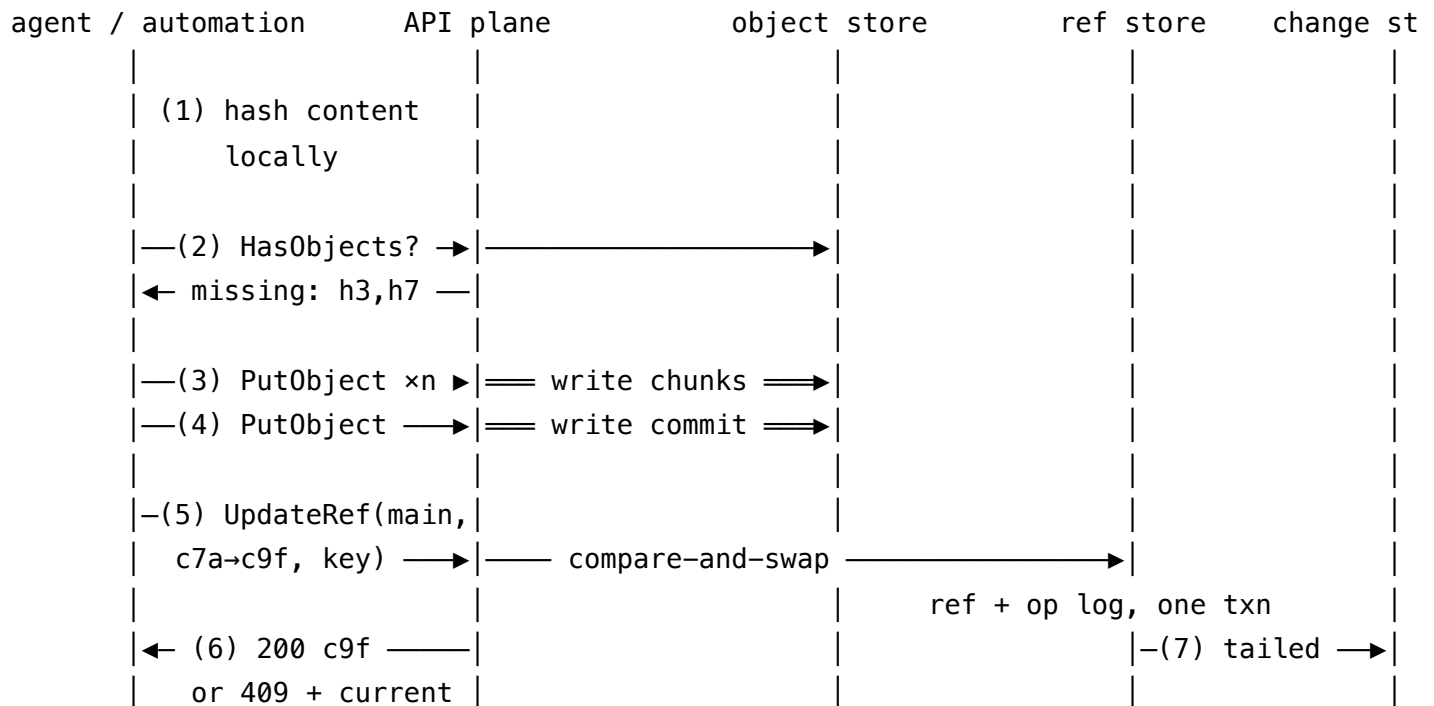
Those invariants land in five components.



- **The API plane is stateless.** Reads scale by adding replicas; writes scale independently.
- **The two stores are independent.** The object store knows nothing about refs; the metadata store holds no content. Degrading one cannot corrupt the other.
- **Every worker is off the request path.** Builds, GC and replication react to events *after* a commit succeeds.

4.4 Request lifecycle

Writing a commit



- **(2) and (3) move only bytes Ledger has never seen.**
- **(3) and (4) strictly precede (5)** — that ordering *is* invariant I3. A crash before (5) leaves unreferenced objects for garbage collection (GC) to reclaim and nothing is visible to anyone.
- **(5) is the system's only atomic point:** it writes the ref and its operation-log entry in one transaction. When two automations race `main`, one wins; the other gets `409` with the current commit and can rebase and retry.
- **(7) tails that log, it is not a second write.** Builds, GC and replication react to events *after* a commit succeeds.

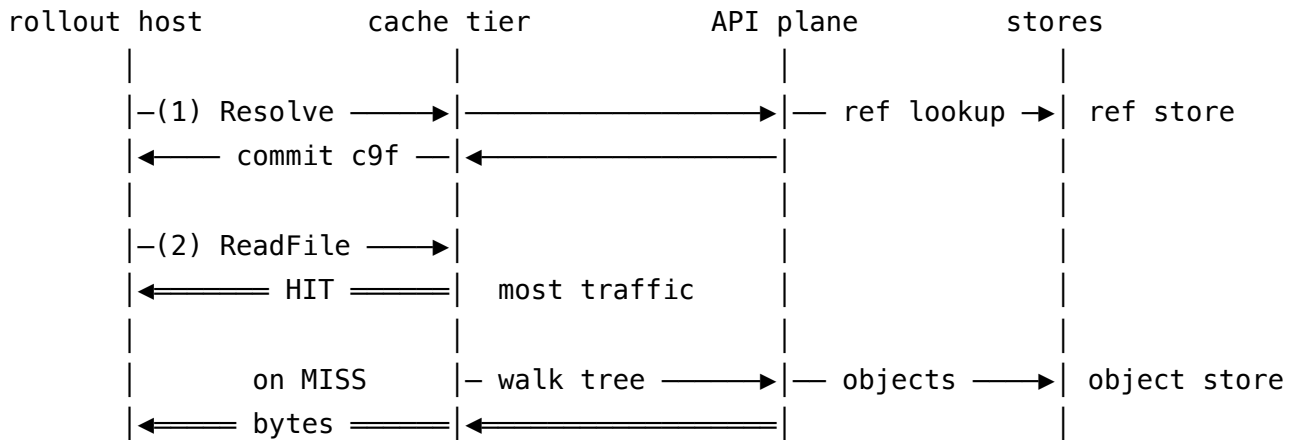
Idempotency - so a retry cannot duplicate

Steps (1)–(4) need no protection as `PutObject` is a no-op the second time. Only step (5) is a state transition.

so every mutating call must carry a client-generated key (with a 24-hour TTL). A repeat returns the recorded outcome; the same key with a different payload returns `422`.

Steps (2)–(5) run inside a write session — a renewable lease (typically an hour) so everything uploaded under it is retained until the session ends.

Reading and materializing



- **Step (1) is the only part that touches mutable state, and it returns a hash.** Everything after it is addressed by content, so it is cacheable forever at every tier supporting 14,000 reads/s.
- **Reads never consult the write path.** The 99% read-only corpus stays readable even when the ref store is degraded.

For bulk reads the API plane does not proxy the bytes. It returns a short-lived signed URL keyed by content hash and the CDN serves the data directly. **The service resolves; the edge delivers.** That keeps the API plane off the bandwidth path entirely

4.5 What Ledger is not

Ledger is deliberately **not** a git server — we do not serve the git wire protocol — nor a code-review product, nor a general CI system.

Section 5 describes the objects in the immutable store. Section 6 describes the mutable names (environments and refs) that point at them.

5. Object model: the immutable half

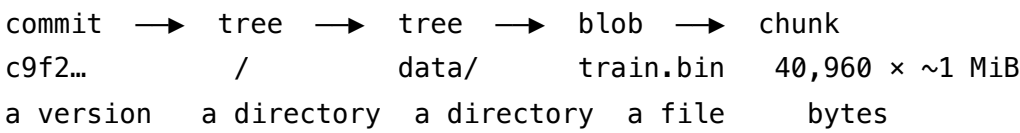
5.1 Objects

An **object** is an immutable byte string whose name is derived from the hash of its own bytes. Objects name are global and not scoped to an environment making identical content shared across environments and forks free.

There are four types:

Type	Represents
Chunk	a slice of a file's bytes
Blob	one file
Tree	one directory
Commit	one version of an environment

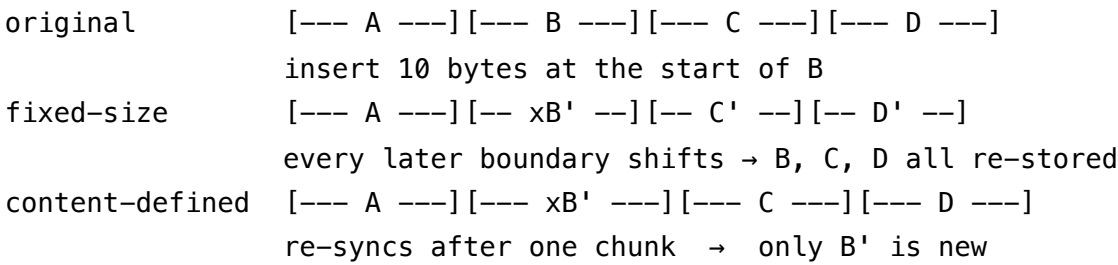
They stack in one direction only:



5.2 Chunks — the unit of deduplication

A chunk is a slice of a file's bytes, averaging ~1 MiB with a 256 KiB minimum.

Making a small edit to a 40 GiB dataset cheaply depends on how files are split. Ledger uses **FastCDC** - a rolling gear hash consumes the file byte by byte and declares a boundary wherever the hash matches a mask. An insertion re-chunks only the region around it and leaves the rest of the dataset byte-identical as shown below:

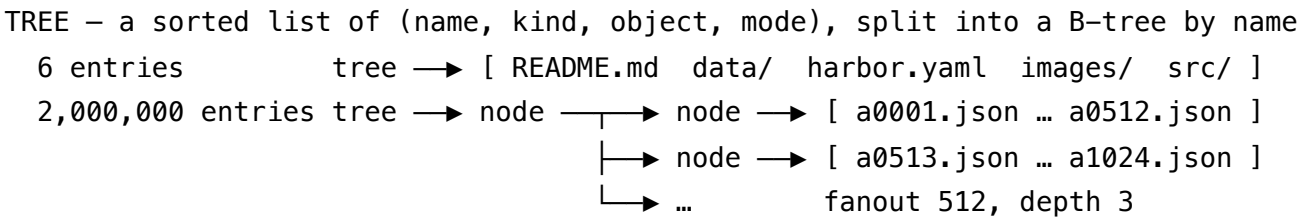
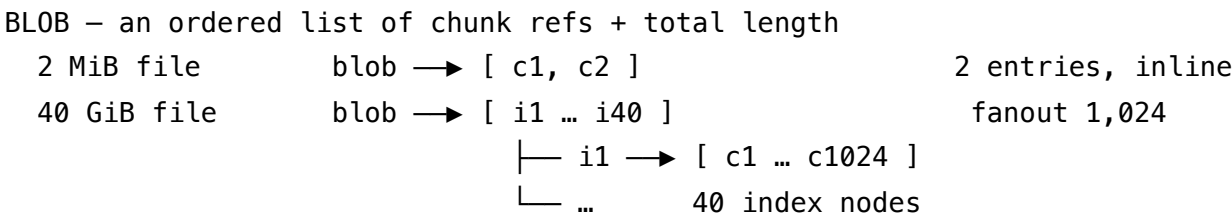


Chunks are stored zstd-compressed but **the name is the hash of the uncompressed bytes**.

5.3 Blobs and trees — no size limit, no width limit

Chunks solve editing a large file, but two problems still remain are handling a file too large for one list of chunks and a directory too wide for one object.

Both take the same approach — stay flat while small and split into a shallow index when large as shown below:



Every blob entry carries its chunk's length, so a node knows the byte range it covers and a ranged read **descends** to the right chunk instead of scanning.

Every tree is ordered by name, so a listing is a **range scan** with a stable cursor.

Structure	Scale	Depth	Read one item	Add one item
Blob	≤ 1 GiB (1,024 chunks)	1	2 fetches	rewrite manifest
Blob	40 GiB → 1 TiB	2	3 fetches	rewrite 1 index node + manifest
Tree	≤ 512 entries	1	1 fetch	rewrite 1 node
Tree	≤ 262,144 entries	2	2 fetches	rewrite 2 nodes
Tree	≤ 134,000,000 entries	3	3 fetches	rewrite 3 nodes

Git by contrast, stores a directory as one flat object — a 2M-entry directory becomes one enormous object rewritten in full for any small change.

5.4 Commits - a snapshot

Trees give a directory while a commit gives a version.

A commit is a complete snapshot — so comparing two versions and going back to an earlier history are all ordinary reads.

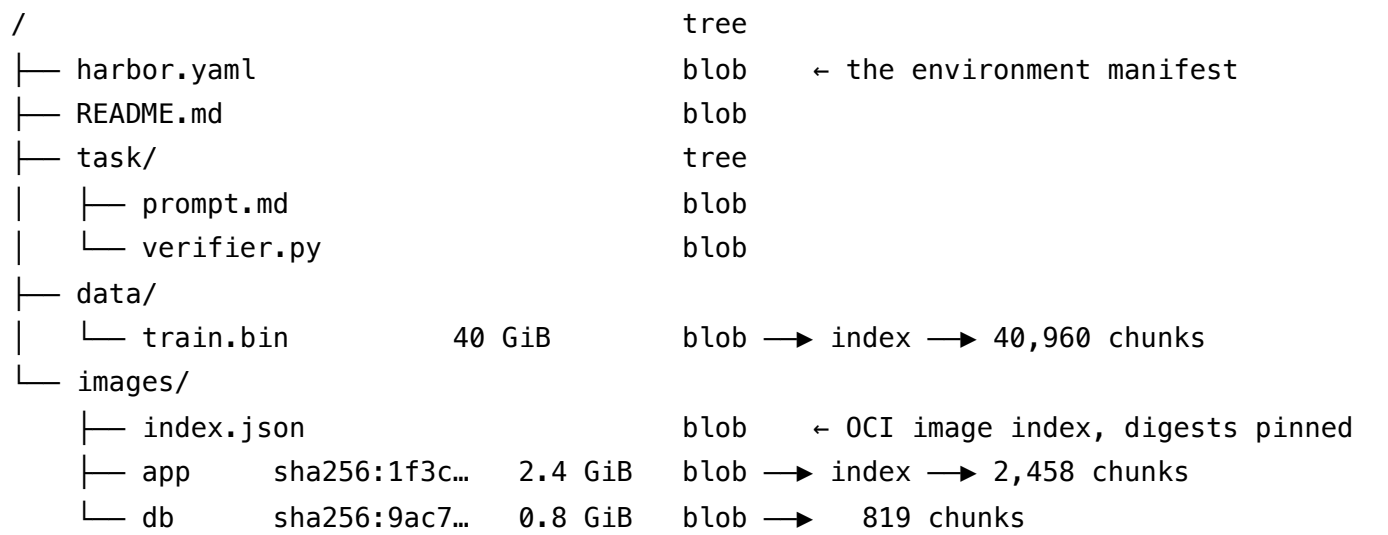
Field	Holds
tree	root tree — the entire content of this version
parents	0 for a root, 1 normally, 2+ for a merge
change_id	stable identity that survives rewrites
author , committer	the principal that produced it

A commit does not name its environment which is what lets a fork share every ancestor object instead of copying it. It carries two identities:

- `commit_id` is the content hash and moves whenever anything changes
- `change_id` is assigned once and survives rewrites. So an automation can name "the change that adds the verifier" across a rebase instead of chasing a hash.

5.5 A whole environment - end to end

Here is what an RL environment looks like:

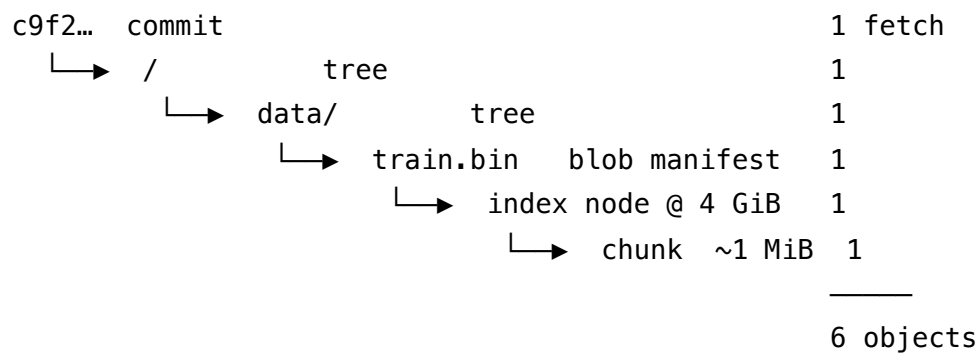


Container images are ordinary content. Each OCI layer is chunked like any other large file and `images/index.json` pins them by digest. The commit *contains* the image bytes rather than pointing at a registry — so restoring an old commit reproduces byte-identical layers even if the tag has moved.

5.6 Reading a file

`ReadFile(commit, path, range)` walks the path server-side and returns just that range — bytes for small reads, signed CDN URLs to the chunks covering it for bulk.

```
ReadFile(c9f2..., "data/train.bin", bytes 4 GiB ... 4 GiB + 1 MiB)
```



Six objects to read for a megabyte out of the middle of a 40 GiB file and the same read against a 1 TiB file costs seven.

6. Environments and refs: the mutable half

Section 5 described the objects in the immutable store. This section describes the mutable names (environments and refs): which environments exist and which commit each ref points at.

6.1 Environments

An environment has: an immutable **id** assigned at creation and a **name** — `org/environment` unique and mutable.

Field	Holds
env_id	Immutable id — the real identity
name	org/environment unique, mutable
labels	Key/value pairs — family=swe-bench
default_ref	Which ref is "the" version — refs/heads/main
forked_from	Parent env_id

Field	Holds
state	active · ready · archived

state indicates whether the environment is ready or is still being worked on.

6.2 Refs

A ref maps (env_id, name) to a commit — the entire mutable surface of a versioned environment and **moving it is the only way to publish a change** — the single atomic point in Ledger.

Field	Holds
env_id	Partition key
name	refs/heads/main
target	Commit object name
generation	Counter, incremented on every successful update
lifecycle	permanent or ephemeral with expires_at

Updates are compare-and-swap and they are compared on **generation not on target** for the following reason:

```

writer A reads   main = c7a1, generation 41
writer B commits main = c9f2, generation 42
writer B reverts main = c7a1, generation 43

compare on target    A expects c7a1, finds c7a1
                    → succeeds, silently erasing 42 and 43
compare on generation A expects 41, finds 43
                    → 409, A re-reads and retries

```

6.3 The metadata store

Everything mutable about one environment lives in one partition - keyed by env_id :

PK = env_01J8XQ7M...	one partition per environment
├─ SK = env	the environment record
├─ SK = ref#refs/heads/main	target, generation, lifecycle
├─ SK = chg#k7m4...	change_id → its commits, so jj can resolve a change
├─ SK = note#c9f2...#qa	QA verdict / rollout result
├─ SK = op#0000000123	operation log entry
└─ SK = idem#7f3a...	outcome of an idempotency key

publish a version = ref + chg + op, ONE single-partition transaction
 another environment = another partition

Since everything for an environment is co-located moving a ref, recording the change and appending to the operation log are **one single-partition transaction** and guarantee atomicity.

DynamoDB, FoundationDB or Postgres partitioned by env_id would all work and must provide:

- A conditional write on one item
- A transaction within one partition
- Strongly consistent single-item reads so Resolve never returns a commit older than one, this caller just wrote.

6.4 Forks and branches

	Fork	Branch
Is	A new environment from an existing commit	A second ref inside one environment
Costs	1 env row + 1 ref row, 0 bytes	1 ref row, 0 bytes
Lives	Indefinitely	Until deleted or past expires_at
On delete	—	Unshared content reclaimed

Neither copies bytes - both are one transaction against one partition.

A fork writes one environment record and one ref in a single transaction.

- A commit to the fork touches only the fork's partition
- And since retention is per-ref over globally shared objects, archiving the parent does not affect the fork.

7. Storage and garbage collection

Sections 5 and 6 defined the immutable objects and mutable names (environments and refs) that point at them. This section is where they physically live: how an object is stored and located.

7.1 Two ways an object is stored

Objects can be stored in cloud object store — S3 or equivalent. Objects take one of two paths - decided purely by size.

```
PutObject(hash, bytes)
|
| — ≥ 256 KiB → standalone at   obj/9f/2c/9f2c8a...
|                  no index entry needed – the name IS the address
|
| — < 256 KiB → appended to an open pack
|                  pack_7b3e... @ offset 41,208, length 2,104
|                  needs an index entry: hash → (pack, offset, length)
```

The standalone path is the one that matters at scale.

- With a 256 KiB chunk minimum, nearly every chunk of every large file lands there — and a standalone object's key *is* its hash, so **locating it takes no lookup at all**.
- Small objects (commits, trees, manifests) — are batched into 64 MiB **packs**. Locating one takes an extra hop: an index entry gives the pack, offset and length.

7.2 Garbage collection

An object must be retained exactly while it is reachable from a live ref and everything else is garbage. The question is how to find it across 21 billion objects without ever deleting one in use.

Why not reference counting: under global dedup a single chunk can be referenced by thousands of commits and every commit would update a counter for each object it touches — means thousands of counter updates per commit. *That adds a distributed-correctness problem directly on the commit path.*

Ledger runs **epoch-based mark and sweep with per-environment keep-sets** - following below four steps:

- (1) freeze ignore everything written after cutoff T — a commit landing mid-cycle is considered next cycle instead, so GC never races a writer
- (2) keep list union the per-environment keep-sets — what each environment's refs and retained op log reach
- (3) compare both lists split into 4,096 shards by hash prefix, so shard n on one side can only match shard n on the other
- (4) sweep standalone object in no keep-set → delete the key, it is the address
packed object → repack or drop the pack once its live bytes fall too

Two things make this affordable:

- **Keep-sets are maintained, not recomputed.** Objects are immutable (l2), so the set of objects a commit reaches is fixed the moment it is written — UpdateRef only adds what that session uploaded and GC never crawls 500M commits.
- **The diff is exact.** Every write appends to a **write catalog**, so the stored side is read from Ledger's own log rather than enumerated out of 21B object-store keys.

8. Platform surfaces

8.1 Authorization

Every path so far has assumed the caller is allowed. Access has two axes — **which operations** and **which environments** and the following construct covers both:

```
grant = principal × { operations } × { environments }

qa-bot × { env:read, env:annotate } × { proximal/swe-bench-* }
```

where a principal is a human on SSO, a long-lived automation or a per-job identity. Operations are coarse scopes:

Operation	Covers
env:read	Resolve, ReadFile, ListDir, Diff, Log, Materialize
env:write	BeginWrite, PutObject, UpdateRef, CreateRef, DeleteRef
env:annotate	Write notes — QA verdicts, rollout results — without access to content
env:fork	Create a new environment starting from a commit in this one

Operation	Covers
env:create	Create a new environment in an org namespace
env:build	Trigger build and sync
env:admin	Rename, archive, change policy

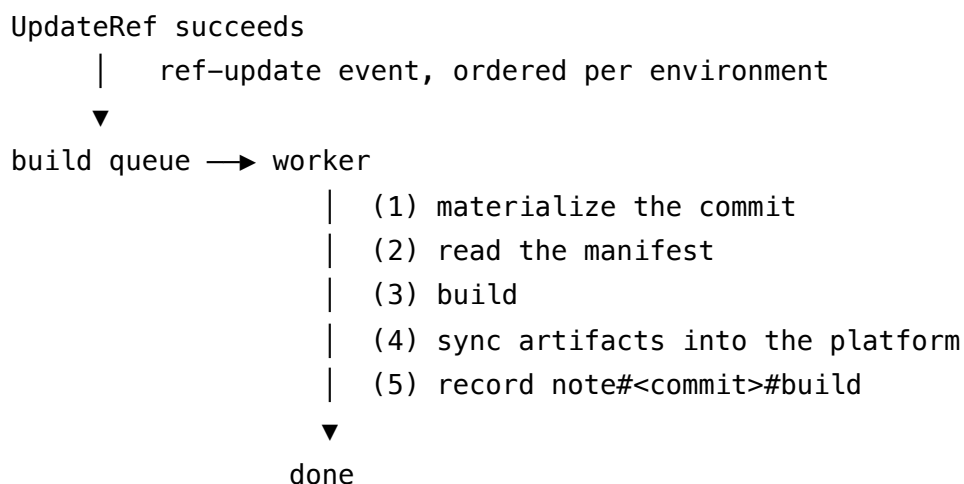
- Environments are selected by id or by name prefix, so one grant covers a single environment or multiple.
- **Grants are additive and there are no deny rules** — a grant can only widen access.

Enforcement is a signature check, not a policy lookup.

- Selectors resolve once, when a token is minted, so the token itself carries its operations, environments and expiry — on the request path the API plane verifies a signature and never reads policy.
- A token can also be handed on in narrowed form: say a rollout host gets `env:read` on the one environment it is running, for as long as that rollout lasts.

8.2. Build and sync

Today every environment carries its own GitHub Action that builds it and syncs data into the platform — 10M copies of nearly the same workflow, each free to drift. Ledger replaces all of them with one centrally versioned pipeline.



A build is a pure function of a commit. A commit captures content rather than pointing at storage that can change, so a build's inputs are fixed forever.

8.3. Client surface

Ledger is reached through four surfaces:

Caller	Surface
Generation agents, humans	jj , with a Ledger backend
Background automations	HTTP API
Container runtimes	OCI registry endpoint, read-only
Rollout hosts	Read-only mount, materialized on demand

jj already separates its user interface from its storage and Ledger is implemented as a **jj backend**. So agents can run real jj where commits, trees and files are Ledger's objects.

9. Operating it

9.1. Observability

Each of the following needs to be instrumented and checked continuously:

Assumption	Metric	Investigate when
≥95% of reads absorbed by cache before the object store	hit ratio per tier	below 90%
Uploads are mostly deduplicated away	objects uploaded ÷ objects offered	above ~20%
~2 GiB unique bytes per environment	unique bytes ÷ environments	above ~3 GiB
Automated writers rarely collide	409 rate per environment	one environment dominating
Builds take ~5 minutes	build duration p50 / p95	p50 above ~10 min
Reclamation keeps pace	sweep lag, garbage backlog	backlog trending up

9.2. Cost

Every storage number scales linearly with one input — unique bytes retained per environment across its 50-version life.

Unique bytes per environment	Corpus	Chunks	Storage cost/month
0.5 GiB	5 PiB	5.4B	~\$76K
2 GiB (assumed)	20 PiB	21B	~\$305K
8 GiB	80 PiB	84B	~\$1.22M

A 4× jump in that estimate moves the dominant cost 4× and moves nothing else materially: request rates, the metadata store, the index and the cache tiers are all governed by traffic rather than corpus size.

The table below list prices at the assumed 2 GiB per environment, excluding compute for 1M concurrent rollouts:

	Year 1 (~1M environments)	Year 10 (10M environments)
Object storage, primary region	~\$30K	~\$305K
Cross-region replica	~\$25K	~\$250K
Build workers	~\$70K	~\$70K
Caches, API plane, index	~\$50K	~\$50K
Object-store requests · metadata store · intra-region transfer	~\$9K	~\$9K
Monthly total	~\$185K	~\$685K

9.3 Phasing and open questions

Delivery splits into four phases:

Phase	Delivers	Ledger becomes
1 — Read	Object model, chunking, GitHub import, Resolve / ReadFile / ListDir , cache tier, mount,	A continuously synced replica of GitHub that all

Phase	Delivers	Ledger becomes
	authorization	automations read from
2 – Write	Write path, ref conditional writes, idempotency, operation log, notes, forks and branches, jj backend, GC	Authoritative
3 – Build	Event pipeline, worker pool, platform sync	The replacement for 10M per-environment Actions
4 – Efficiency	Packing, storage tiering, multi-region	Cheaper and more available

Open questions:

1. **Is 2 GiB of unique bytes per environment right?** Every storage number scales with it.
2. **Should OCI layers be stored uncompressed?** Gzip destroys content-defined chunking, so storing the uncompressed tar preserves dedup — but at the price of not reproducing the legacy gzip digest.